



Intro to Coding & TypeScript — Classwork

Your first real code! Work in groups. Type into this sheet as you go.

Group / Name:

Date:

Mission Control — Set Up Your Team (about 3 min)

Today you write REAL TypeScript in your browser. No installing anything — it works on any computer.

- 1 As a group, open this website: typescriptlang.org/play
- 2 Pick a DRIVER — the person who types and shares their screen first. You will ROTATE the Driver every Mission so everyone gets a turn.
- 3 Delete any sample code in the left box so you start with a blank sheet.

EVERYONE HAS A ROLE

Can't open the site (locked-down computer)? No problem — you become the group's ARCHITECT: you read the instructions aloud and tell the Driver exactly what to type. You're the brains of the operation!

Write your group's Driver rotation order (who goes 1st, 2nd, 3rd...):

Mission A — Make the Computer Talk (about 8 min)

Driver #1 at the keyboard. The command `console.log( )` prints whatever you put inside it.

COPY THIS EXACTLY, THEN PRESS RUN

```
console.log("Hello, world!");  
console.log("Our group is ready to code.");  
console.log(7 + 3);
```

Look at the console (bottom of the screen). Write down EXACTLY what printed on each line:

Line 1 printed:

Line 2 printed:

Line 3 printed (was it `7 + 3` or `10`):

CONCEPT: TEXT VS. NUMBERS

Text goes inside "quotes" and prints exactly as written. Numbers have NO quotes and can be math'd. That's why `7 + 3` without quotes prints 10.

Now make it yours:

Write TWO new `console.log` lines of your own — print your group name and your favorite game. Copy what



you typed here:

Mission B — Variables: Labeled Boxes (about 10 min)

Rotate! Driver #2 takes the keyboard. A variable is a labeled box that stores a value with 'let'.

TYPE THIS AND RUN

```
let playerName = "Alex";  
let score = 0;  
score = score + 10;  
console.log(playerName);  
console.log(score);
```

What two things printed to the console?

It printed:

Change "Alex" to a name your group picks, and change the starting score to any number. Run it again.

Your new name value:

Your new score after adding 10:

CONCEPT: UPDATING A VARIABLE

The line `score = score + 10` means: take the box called `score`, add 10, and put the answer back in the box. Computers read the right side first, then store it on the left.

Mission C — Build a Player Card (with Types!) (about 10 min)

Rotate! Driver #3. This is the TypeScript part — we tell each box what *KIND* of thing it holds.

TYPE THIS STARTER, THEN RUN

```
let name: string = "Nova";  
let level: number = 6;  
let isReady: boolean = true;  
console.log("Player: " + name);  
console.log("Level: " + level);  
console.log("Ready to play? " + isReady);
```

CONCEPT: THREE BASIC TYPES

`string` = text in quotes. `number` = any number. `boolean` = only true or false. These 'types' are the safety helmet that makes TypeScript catch your mistakes early.

Design your OWN player. Fill in the values you'll use, then change them in the code:

name (a string, in quotes):



level (a number):

isReady (true or false):

Add ONE more variable of your own to the player card (for example let power: number = 100;). Write the line you added:

Mission D — Bug Hunt! (about 8 min)

Whole group works together. Real coders spend tons of time DEBUGGING — finding and fixing mistakes.

Type this broken code into the Playground. TypeScript will underline a problem in red. Read the error!

TYPE THIS BUGGY CODE

```
let score: number = 10;  
score = "twenty";  
console.log(score);
```

What is TypeScript complaining about? Write the error (or explain it in your own words):

Now FIX the bug so the code runs with no red underline. What did you change?

Our fix:

RECAP: WHY TYPES HELP

Notice TypeScript caught the mistake BEFORE the program even ran — because we told it score must be a number. In plain JavaScript this bug would sneak through silently.

Mission E — Math Machine (about 8 min)

Rotate the Driver again. Let's do some math and mix numbers with text.

TYPE THIS AND RUN

```
let apples: number = 8;  
let oranges: number = 5;  
let total: number = apples + oranges;  
console.log("Total fruit: " + total);
```

What printed?

Try these mini-challenges. Write the line you used AND the answer that printed:

1) Print how many MORE apples than oranges there are:

2) Make a variable for your age, then print your age in 5 years:



Mission F — Design Challenge: Your Own Program (about 10 min)

Open-ended! As a group, invent a tiny program using variables + console.log. Ideas below.

Pick ONE (or invent your own): a) a Character Generator that prints a hero's name, level, and power; b) a Scoreboard for two players; c) an 'About Me' card for your group.

Our program idea:

Plan your variables first (name and type), then build it. Write your finished code here:

What was the trickiest part, and how did your group solve it?

Quick Quests — Finished Early? Grab More! (any time)

Check off each one your group completes. Fast finishers: don't just sit — quest on!

- ☐ Print a line that combines TWO variables and some text in one message.
- ☐ Make a boolean variable and print a sentence that uses it.
- ☐ Use console.log to print a little 3-line poem or rap about coding.
- ☐ Change a number variable using multiplication (use the * symbol).
- ☐ Give every variable in your player card a helpful, clear name.
- ☐ **STRETCH** Design a program that prints a full 'team roster' with 3 players, each with a name, level, and boolean. Then explain to another group how your types keep it safe.

Talk It Out — Teach a Teammate (about 2 min)

Turn to a partner. In your OWN words (no peeking!), explain what a VARIABLE is and give one example. Then write your explanation here:

Reflect

One thing I can do now that I couldn't at the start of class:

One thing I still have a question about:

How did today feel? (circle one) Too easy Just right Challenging (in a good way) Tricky